Reversing CTF

By: Tehila Chananayev - 325435956

Introduction

The purpose of the assignment is to simulate a real-world cybersecurity challenge, where we must analyze a fancy binary file, understand its logic, and overcome three consecutive steps using techniques learned during the course.

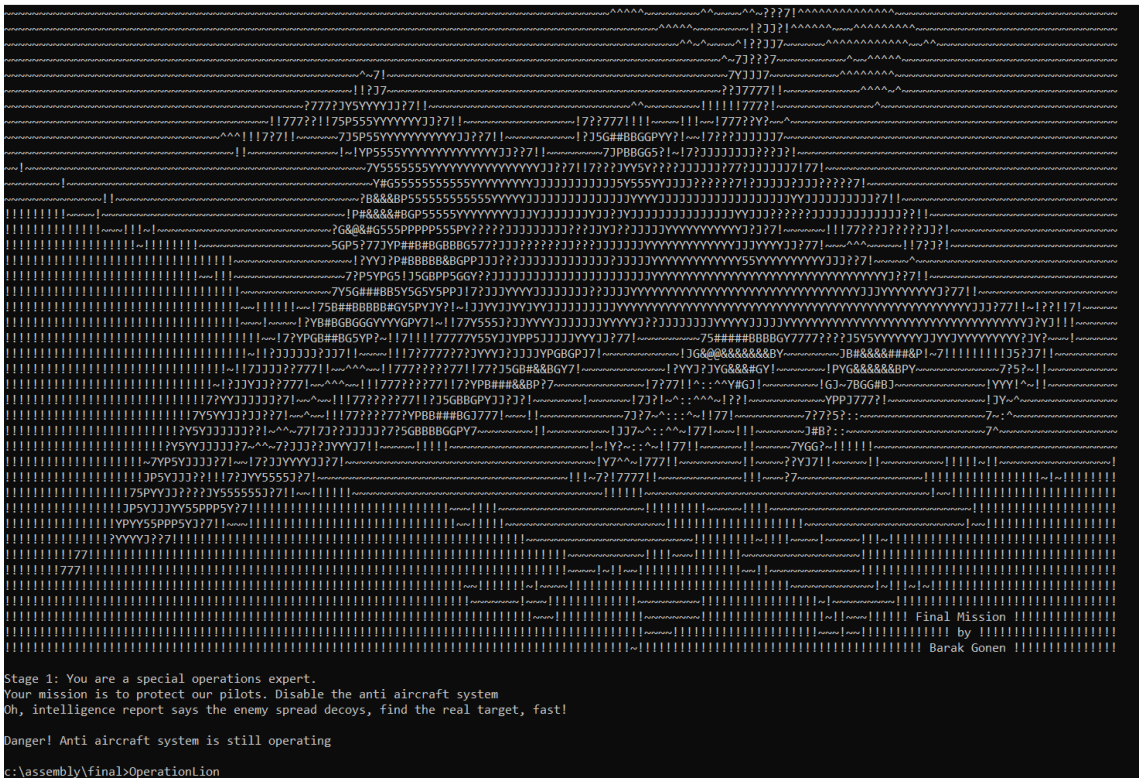
The scenario presented is that we are in the role of special operations specialists, tasked with protecting Israeli pilots by disabling an enemy anti-aircraft system. As part of the mission, we are warned about decoys and must identify the correct target through careful analysis.

This description includes a full explanation of each step, the tools I used, the reverse engineering process, and how I gradually arrived at the correct solution. I have documented each step to emphasize understanding over guesswork, and where possible, I have supported my reasoning with screenshots or code snippets.

I did not use any of the help files on the course website.

Sage1

First of all I ran the program and saw the following output:



Drawing a plane from scratch and printing the following caption

```
Stage 1: You are a special operations expert.
Your mission is to protect our pilots. Disable the anti aircraft system
Oh, intelligence report says the enemy spread decoys, find the real target, fast!

Danger! Anti aircraft system is still operating
```

The scenario presented is that we are in the role of special operations specialists, tasked with protecting Israeli pilots by disabling an enemy anti-aircraft system. As part of the mission, we are warned about decoys and must identify the correct target through careful analysis.

Stage 1: Initial Analysis and Approach

Once I fully understood the task, I began the reverse engineering process. My first step was to open the binary file in IDA, where I began examining the code. At first, I noticed that the program printed a string representing a drawing of an airplane, and then printed another string. This sequence of actions was my first clue that there might be important information hidden in the output. As I continued to analyze the disassembled code, I identified two key functions that seemed to be integral to the logic of the program that took the string Radar as input. And I thought that these functions would actually lead me to the solution.

```
.text:007C17BB xor     eax, ebp
.text:007C17BD mov     [ebp+var_4], eax
.text:007C17C0 mov     eax, off_7C9018 ; "~~~
```

```
unsigned int __cdecl Bait_1(const char *a1)
{
  unsigned int result; // eax
  unsigned int i; // [esp+Ch] [ebp-8h]

  for ( i = 0; ; ++i )
  {
    result = i;
    if ( i >= strlen(a1) )
      break;
    a1[i] ^= 0x2Au;
    a1[i] ^= 0x2Au;
  }
  return result;
}
```

Step 1: Investigation and Hypothesis

After analyzing the initial behavior of the program and the first function, I hypothesized that the purpose of the XOR operation in the Bait_1 function might not be as simple as it seemed. While the function performs an XOR operation twice on each character in the input string, which essentially returns the characters to their original state, I wanted to verify whether this was indeed the case.

To test this, I examined the result of performing the XOR operation on a sample string and found that after a single XOR operation, the result was "Radar=xNKKX". However, this output did not reveal any useful or meaningful information, and I realized that it did not help me move forward with the solution. This led me to conclude that the function might not be the key to deciphering the target information.

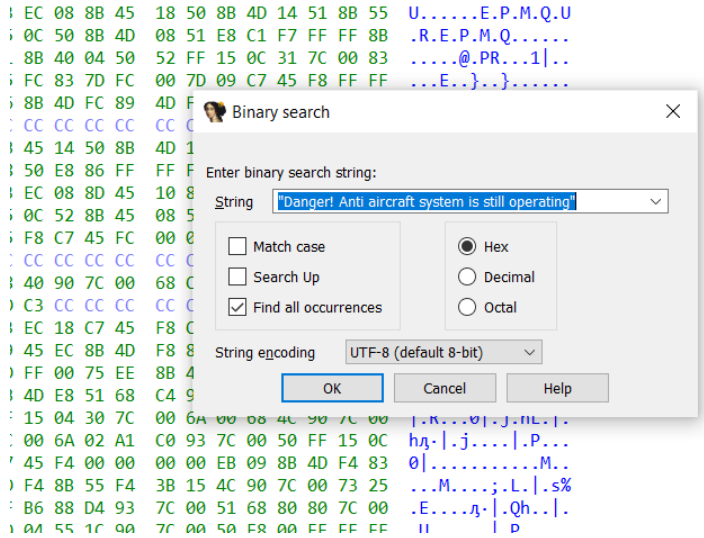
I also considered another potential approach: the string representing the airplane drawing printed earlier in the code. This looked suspicious, and I thought that it might contain hidden data that could be important. I suspected that perhaps performing an XOR operation on this string would yield some useful information, so I prepared to write a small program that would process this string and perform an XOR on it. However, before I continued with that program, I decided to examine the second function in the code. I was hoping it would provide some more insight or direction on how to proceed.

This turned out to be a seductive function to waste my time.

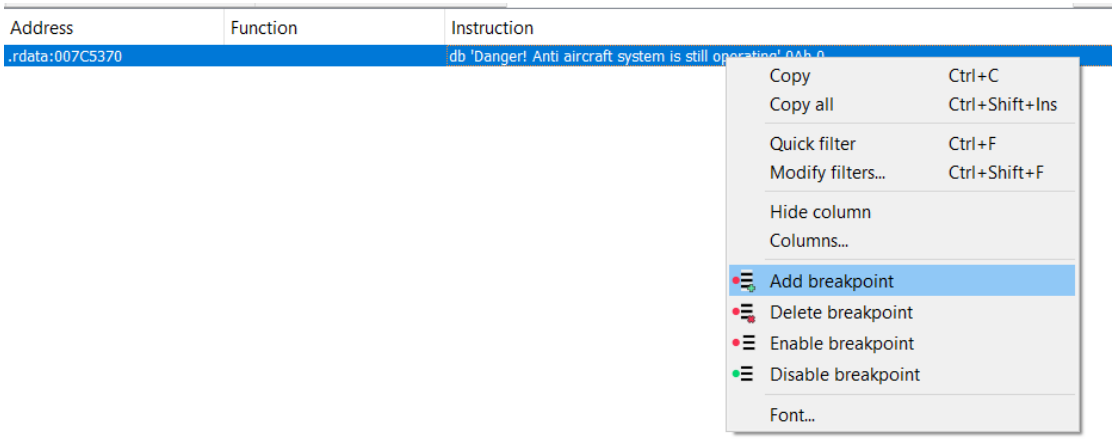
Step 2: Memory Search and Debugging

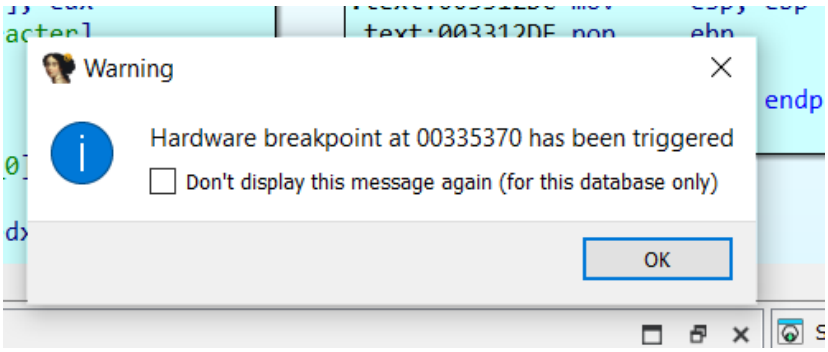
After reviewing the program output further, I noticed the string**: "Danger! Anti-aircraft system still running"**. So I thought I would search for the string and see what was causing this error, i.e. what test failed, but I couldn't find it anywhere in the code itself. I suspected that it might be hidden deeper within the program, perhaps in memory.

To investigate further, I decided to search the program's memory. I used the Alt + B shortcut to open the Memory Search window and entered the string "Danger! Anti-aircraft system still running".



After the search, I set a hardware breakpoint on the string, which allowed me to pause the program when it tried to access this specific memory area.





I then ran the program until the hardware breakpoint was triggered. This allowed me to examine the state of the program at the exact moment it was about to display the message. By following the steps using Ctrl + F7, I was able to follow the flow of the program and understand what led to the message being printed.

After identifying the code segment responsible for printing the message, I began to investigate the program in depth to determine what actions were required to disable the anti-aircraft system.

I saw that there was a string that looked like a path.

```
.text:00331440 mov     [ebp+var_4], eax
.text:00331443 push    esi
.text:00331444 push    edi
.text:00331445 mov     [ebp+var_28], 43h ; 'C'
.text:00331449 mov     [ebp+var_28+1], 3Ah ; ':'
.text:0033144D mov     [ebp+var_28+2], 5Ch ; '\'
.text:00331451 mov     [ebp+var_28+3], 52h ; 'R'
.text:00331455 mov     [ebp+var_28+4], 65h ; 'e'
.text:00331459 mov     [ebp+var_28+5], 76h ; 'v'
.text:0033145D mov     [ebp+var_28+6], 65h ; 'e'
.text:00331461 mov     [ebp+var_28+7], 72h ; 'r'
.text:00331465 mov     [ebp+var_28+8], 73h ; 's'
.text:00331469 mov     [ebp+var_28+9], 69h ; 'i'
.text:0033146D mov     [ebp+var_28+0Ah], 6Eh ; 'n'
.text:00331471 mov     [ebp+var_28+0Bh], 67h ; 'g'
.text:00331475 mov     [ebp+var_28+0Ch], 43h ; 'C'
.text:00331479 mov     [ebp+var_28+0Dh], 54h ; 'T'
.text:0033147D mov     [ebp+var_28+0Eh], 46h ; 'F'
.text:00331481 mov     [ebp+var_28+0Fh], 5Ch ; '\'
.text:00331485 mov     [ebp+var_28+10h], 0
.text:00331489 mov     [ebp+var_14], 44h ; 'D'
.text:0033148D mov     [ebp+var_14+1], 72h ; 'r'
.text:00331491 mov     [ebp+var_14+2], 6Fh ; 'o'
.text:00331495 mov     [ebp+var_14+3], 6Eh ; 'n'
.text:00331499 mov     [ebp+var_14+4], 65h ; 'e'
.text:0033149D mov     [ebp+var_14+5], 41h ; 'A'
.text:003314A1 mov     [ebp+var_14+6], 74h ; 't'
.text:003314A5 mov     [ebp+var_14+7], 74h ; 't'
.text:003314A9 mov     [ebp+var_14+8], 61h ; 'a'
.text:003314AD mov     [ebp+var_14+9], 63h ; 'c'
.text:003314B1 mov     [ebp+var_14+0Ah], 6Bh ; 'k'
.text:003314B5 mov     [ebp+var_14+0Bh], 2Eh ; '.'
.text:003314B9 mov     [ebp+var_14+0Ch], 74h ; 't'
.text:003314BD mov     [ebp+var_14+0Dh], 78h ; 'x'
```

The path string **C:\ReversingCTF\DroneAttack.txt** has been constructed and is used within the code. The program uses this file path to check if the file exists. It first creates the path string by concatenating "C:\ReversingCTF\" with "DroneAttack.txt". The individual characters of this path are copied byte by byte from memory into the FileName variable.

```
.text:003315AC mov     edi, [ebp+var_2B8]
.text:003315B2 mov     esi, [ebp+var_2D0]
.text:003315B8 mov     eax, [ebp+var_2D4]
.text:003315BE mov     ecx, eax
.text:003315C0 shr     ecx, 2
.text:003315C3 rep movsd
.text:003315C5 mov     ecx, eax
.text:003315C7 and     ecx, 3
.text:003315CA rep movsb
.text:003315CC lea     ecx, [ebp+FindFileData]
.text:003315D2 push    ecx             ; lpFindFileData
.text:003315D3 lea     edx, [ebp+FileName]
.text:003315D9 push    edx             ; lpFileName
.text:003315DA call    ds:FindFirstFileA
.text:003315E0 mov     [ebp+hFindFile], eax
.text:003315E6 cmp     [ebp+hFindFile], 0FFFFFFFFh
.text:003315ED jz      loc_33177A
```

This code snippet checks if a file exists in the path given to it (C:\ReversingCTF\DroneAttack.txt). It does this by using the FindFirstFileA function of the WinAPI, which performs a file search File Management. If the function is able to find the file, it returns information about the file, and if not, the program stops searching.

If the file does not exist (if FindFirstFileA returns an invalid value), then the program prints :the message

"Danger! Anti aircraft system is still operating\n"

```
.text:003315F3 push    7D0h            ; dwMilliseconds
.text:003315F8 call    ds:Sleep
.text:003315FE push    offset aAntiAircraftSy ; "Anti aircraft system located\nIntiating"...
.text:00331603 call    print_slowly
.text:00331608 add     esp, 4
.text:0033160B mov     eax, [ebp+hFindFile]
.text:00331611 push    eax             ; hFindFile
.text:00331612 call    ds:FindClose
.text:00331618 push    0               ; hTemplateFile
.text:0033161A push    80h             ; dwFlagsAndAttributes
.text:0033161F push    2               ; dwCreationDisposition
.text:00331621 push    0               ; lpSecurityAttributes
.text:00331623 push    0               ; dwShareMode
.text:00331625 push    40000000h       ; dwDesiredAccess
.text:0033162A lea     ecx, [ebp+FileName]
.text:00331630 push    ecx             ; lpFileName
.text:00331631 call    ds:CreateFileA
.text:00331637 mov     [ebp+hFile], eax
.text:0033163D cmp     [ebp+hFile], 0FFFFFFFFh
.text:00331644 jz      loc_331760
```

In this part, the Create FileA function, also from winAPI, verifies that the file was indeed opened successfully.

At this point, I understood that in order to disable the system and progress to the next stage, I needed to create the required file in the specified location. The program checks for the existence of the file C:\ReversingCTF\DroneAttack.txt. If the file is found, the system will proceed to disable the anti-aircraft system and print the message:

Great job. Anti aircraft system is disabled

To proceed, I manually created the file DroneAttack.txt in the folder C:\ReversingCTF\, and upon running the program again, the system was successfully disabled, allowing me to move on to **Stage 2**.

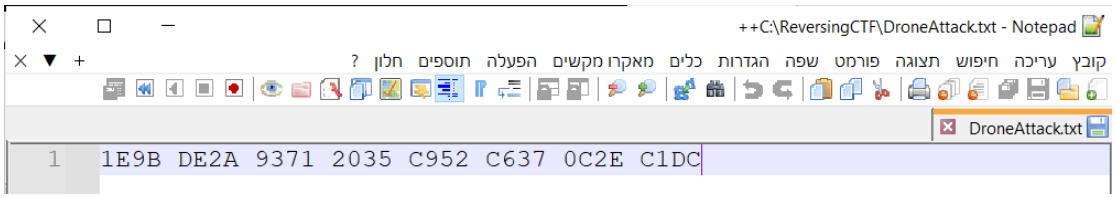
```
Stage 1: You are a special operations expert.
Your mission is to protect our pilots. Disable the anti aircraft system
Oh, intelligence report says the enemy spread decoys, find the real target, fast!

Anti aircraft system located
Intiating disable sequence

Great job. Anti aircraft system is disabled

Stage 2: You are a jet fighter pilot. The sky is clear. Your mission: release bombs on IRGC headquarters.
To find them, use the cyber intelligence unit
```

Sage2

This is the next task.

```
Stage 2: You are a jet fighter pilot. The sky is clear. Your mission: release bombs on IRGC headquarters.
To find them, use the cyber intelligence unit
```

After step 1 in the file I created the following content is recorded

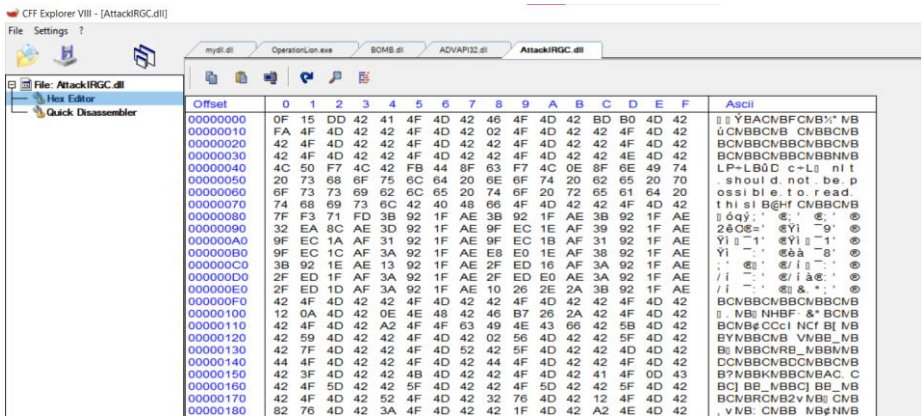


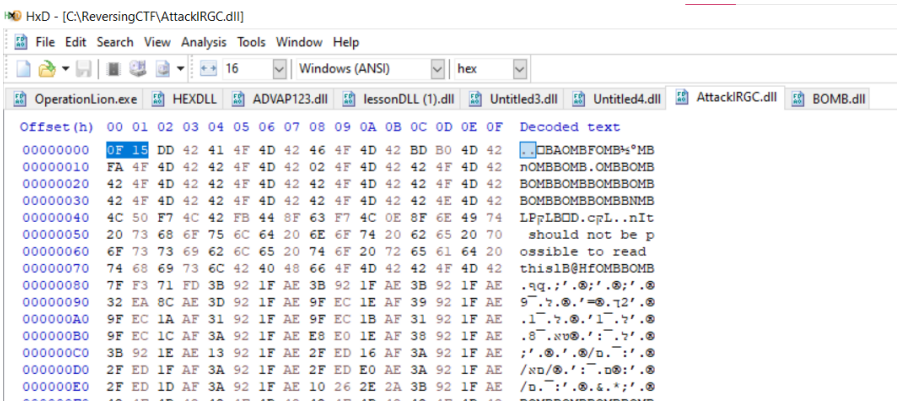
And a DLL file was also created in the ReversingCTF folder called AttackIRGC.dll

first of all, I knew I needed to check and load the DLL file to make it do something.

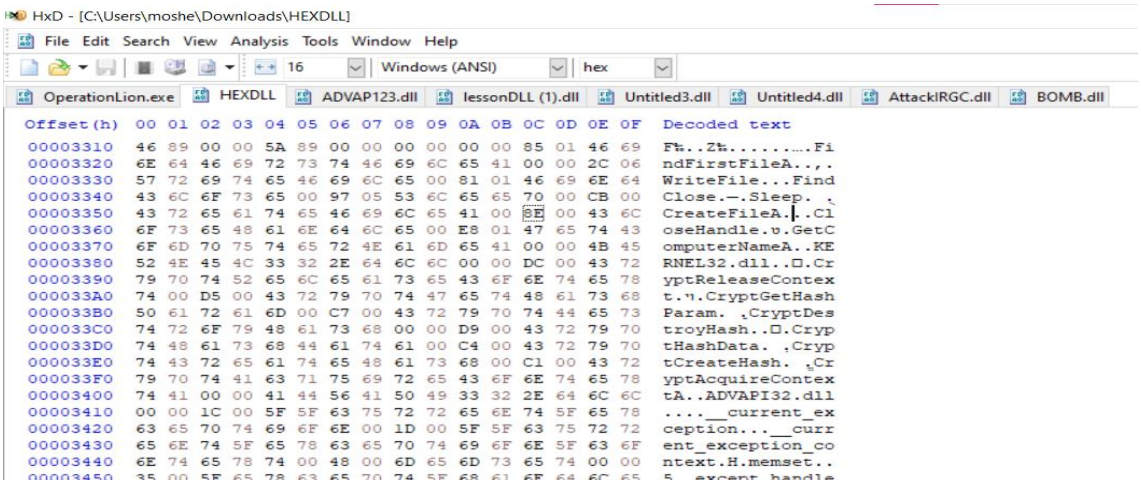
Investigation and Hypothesiss:

I put it in CFF Explorer and also in H&D to first investigate what I was seeing and saw that it was not in PE (Portable Executable) format, which is the standard format for executable files in Windows. The MZ header, which usually marks the beginning of an EXE or DLL file, was missing. This led me to the conclusion that it needed to be fixed manually or with the help of additional tools.

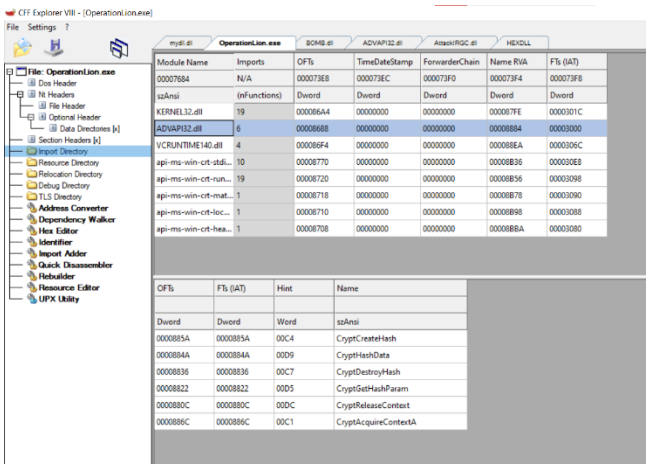




I thought about making changes to the DLL file using H&D and manually changing it to get the PE format. I added the MZ header at the beginning along with the full header, but that didn't work either. I searched the file and saw that there were no .TEXT or .DATA sections, so I realized that this was no longer the solution. I thought maybe I could take the data that goes into the DLL in HEX format from the program and play with it. I even considered creating a wrapper for the existing DLL using C code, but I just got stuck.



So when I saw that this was not the case and I was getting into trouble, I looked again at the task of stage 2 "To find them, use the **cyber intelligence unit**" - I realized that I needed to use cyber, meaning something encrypted that needed to be decrypted, etc. Then it occurred to me that maybe the DLL file was encrypted and the key was the characters that were entered in the program for the file I created in stage 1, so what encryption could it be? Encrypting with AES doesn't make sense because you also need an IV. The best thing that worked for me was XOR, which isn't too complex. Then I decided to try it (I asked the chat if there was a site that decrypted it or I would make some code in Python). I inserted the file into an online site that decrypted it for me with the key and the key wasn't correct. I tried a few more strings related to work. I tried ReversingCTF, IRGC and it didn't work.



In addition, I looked at the OperationLion.exe program in CFF explorer to see what DLL the program uses and what functions, so I later changed the name of the DLL after decoding to ADVAPI23.dll.

A way to a solution

Then I thought I need, the beginning of the file should be in PE format, so there is the string MZ, which is 4D 5A HEX at the beginning, so I looked at the characters at the beginning of the DLL file, the characters were OF 15, so I thought what would give me MZ

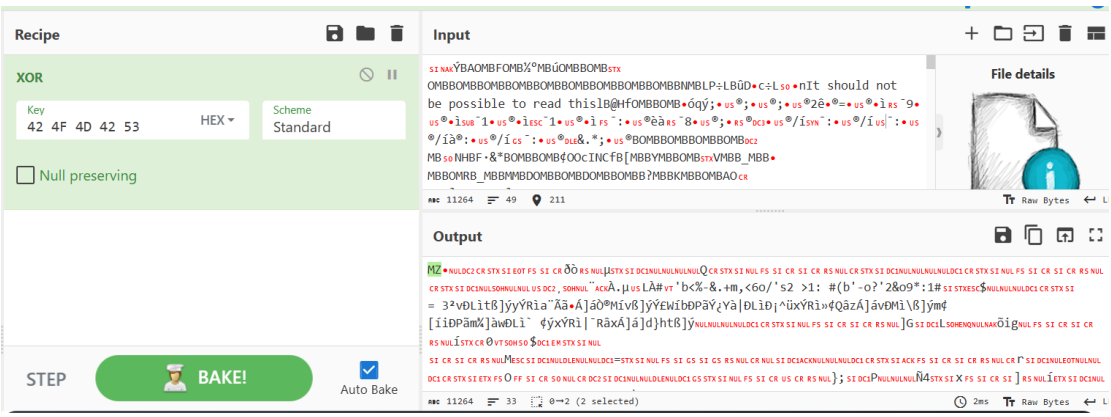
0F XOR X = 4D

15 XOR X = 5A

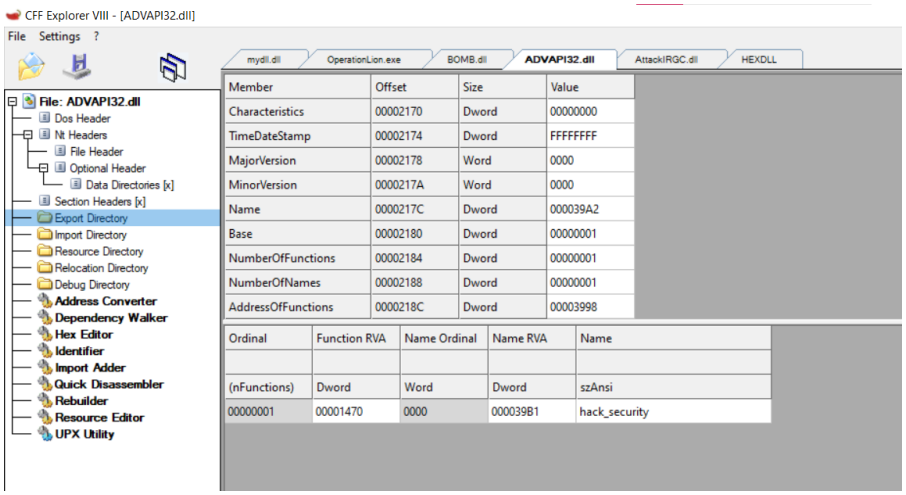
X = 0F XOR 4D = 42 = 'B'

X = 15 XOR 5A = 4F = 'O'

Therefore, the beginning of the key should start with **'BO'**. I thought about what could be that starts with these characters and is related to the mission, and I looked again at the mission "Your mission: Drop **bombs** on the Revolutionary Guard headquarters" I tried the key that equals **"BOMB"** and indeed that was the key.



After I broke the encryption, I opened the DLL file in CFF Explorer to see what was in it.



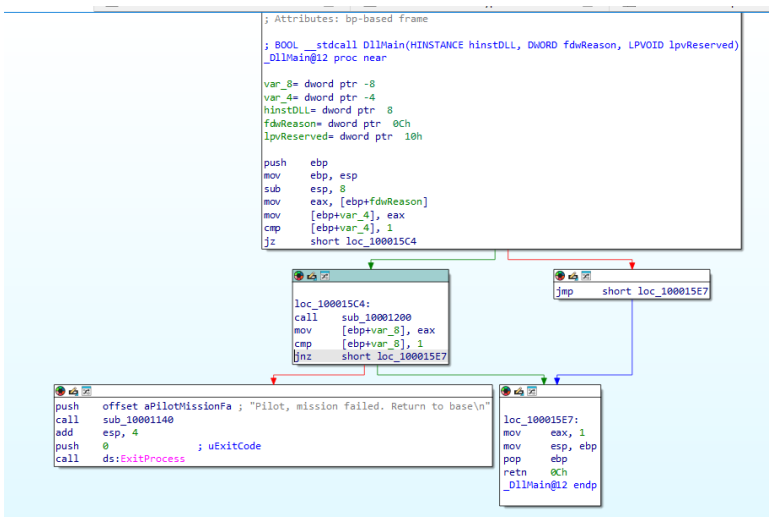
So I wrote some C code in Visual Studio to load the DLL into the program

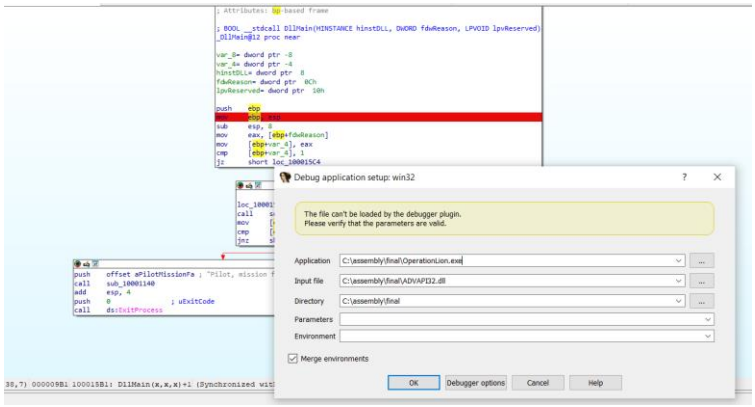
(I took the Operating Systems Run Time Tutorial in DLL Import 8.3.3)

I just changed the path of the DLL and the function to be hack_securty

```
lodeDLL                                                    (Global Scope)
1   #include <windows.h>
2   #include <stdio.h>
3
4   #define DLL_PATH "C:\\assembly\\final\\ADVAPI32.dll" // הכנס את הנתיב המלא של ה-DLL שלך
5
6   typedef void (*PFUNC)(void);  // הגדרת טיפוס לפונקציה ב-DLL
7
8   int main()
9   {
10      // בזיכרון ה-DLL טעינת
11      HMODULE hModule = LoadLibraryA(DLL_PATH);
12      if (hModule == NULL) {
13          printf("Failed to load DLL: %lu\n", GetLastError());
14          return 1;
15      }
16
17      // חיפוש הפונקציה בתוך ה-DLL
18      PFUNC pFunc = (PFUNC)GetProcAddress(hModule, "hack_security");
19      if (pFunc == NULL) {
20          printf("Failed to get function address: %lu\n", GetLastError());
21          FreeLibrary(hModule);  // שחרור ה-DLL אם לא נמצאה הפונקציה
22          return 1;
23      }
24
25      // קריאה לפונקציה ב-DLL
26      (*pFunc)();
27
28      // שחרור ה-DLL אחרי השימוש
29      FreeLibrary(hModule);
30
31      return 0;
32  }
33
```

I opened the DLL in IDA.





I ran it and it was printed for me -"Pilot, mission failed. Return to base"

I investigated the code in static analysis and saw that there was anti-debugging.

The function checks to see if the process is being debugged by accessing the BeingDebugged field within the PEB (Process Environment Block). If the value in BeingDebugged is 1, this indicates that debugging is enabled, and this is considered anti-debug.

```
; Attributes: bp-based frame

; int sub_10001200()
sub_10001200 proc near
push    ebp
mov     ebp, esp
push    ebx
mov     eax, 30h ; '0'
mov     eax, fs:[eax]
xor     ebx, ebx
mov     bl, [eax+2]
mov     eax, ebx
pop     ebx
pop     ebp
retn
sub_10001200 endp
```

IDA View-A | Pseudocode-A

```
1 int sub_10001200()
2 {
3   return NtCurrentPeb()->BeingDebugged;
4 }
```

I went into the TLS section of IDA in the Exports tab and saw the code that brings up the success message, and to get to it, the function argument should be 8200.

IDA View-A | Hex View-1 | Local Types | Imports | Exports

Name	Address	Ordinal
hack_security	10001470	1
DllEntryPoint	10001940	[main entry]

```
; Exported entry   1. hack_security

; Attributes: bp-based frame

; int __cdecl hack_security(int)
public hack_security
hack_security proc near

Stream= dword ptr -34h
var_30= dword ptr -30h
var_2C= dword ptr -2Ch
var_25= byte ptr -25h
var_24= byte ptr -24h
var_4= dword ptr -4
arg_0= dword ptr  8

push    ebp
mov     ebp, esp
sub     esp, 34h
mov     eax, ___security_cookie
xor     eax, ebp
mov     [ebp+var_4], eax
cmp     [ebp+arg_0], 2008h
jz      short loc_100014A3
```

```
push    offset aPilotYouMissed ; "Pilot, you missed the target. Use your "...
call    sub_10001140
add     esp, 4
push    0               ; uExitCode
call    ds:ExitProcess
```

```
loc_100014A3:
push    offset aGreatJobPilotB ; "Great job pilot, bombs hit IRGC.\n\n"
call    sub_10001140
add     esp, 4
push    offset aStage3WelcomeC ; "Stage 3: Welcome cyber specialist.\nYou"...
call    sub_10001140
add     esp, 4
push    offset aYourCountryDep ; "Your country depends on your skills. We"...
```

So I changed it to accept an int argument and send it 8200 so that the task would succeed.

```
lodeDLL.cpp
lodeDLL                                   (Global Scope)
1   #include <windows.h>
2   #include <stdio.h>
3
4   #define DLL_PATH "C:\\assembly\\final\\ADVAPI32.dll" // הכנס את הנתיב המלא של ה-DLL שלך
5
6   typedef void (*PFUNC)(int);  // הגדרת טיפוס לפונקציה ב-DLL
7
8   int main()
9   {
10      // בזיכרון ה-DLL טעינת
11      HMODULE hModule = LoadLibraryA(DLL_PATH);
12      if (hModule == NULL) {
13          printf("Failed to load DLL: %lu\n", GetLastError());
14          return 1;
15      }
16
17      // חיפוש הפונקציה בתוך ה-DLL
18      PFUNC pFunc = (PFUNC)GetProcAddress(hModule, "hack_security");
19      if (pFunc == NULL) {
20          printf("Failed to get function address: %lu\n", GetLastError());
21          FreeLibrary(hModule);  // אם לא נמצאה הפונקציה שחרור ה-DLL
22          return 1;
23      }
24
25      // קריאה לפונקציה ב-DLL
26      (*pFunc)(8200);
27
28      // אחרי השימוש שחרור ה-DLL
29      FreeLibrary(hModule);
30
31      return 0;
32  }
```

I ran it without a debugger then I finally passed a stage and managed to blow up the Revolutionary Guards.

```
Great job pilot, bombs hit IRGC.

Stage 3: Welcome cyber specialist.
Your mission : Penetrate the security system of the supreme leader.
The location of the enriched Uranium is stored there.
Your country depends on your skills. We COUNT on you. Good luck.
Enter code
```

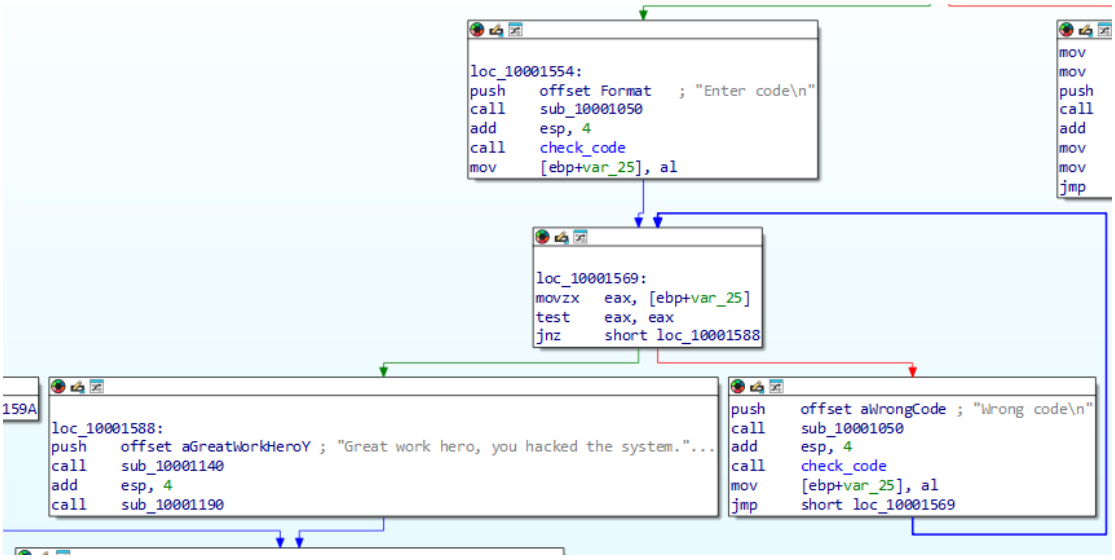
Sage3

This is the next task.

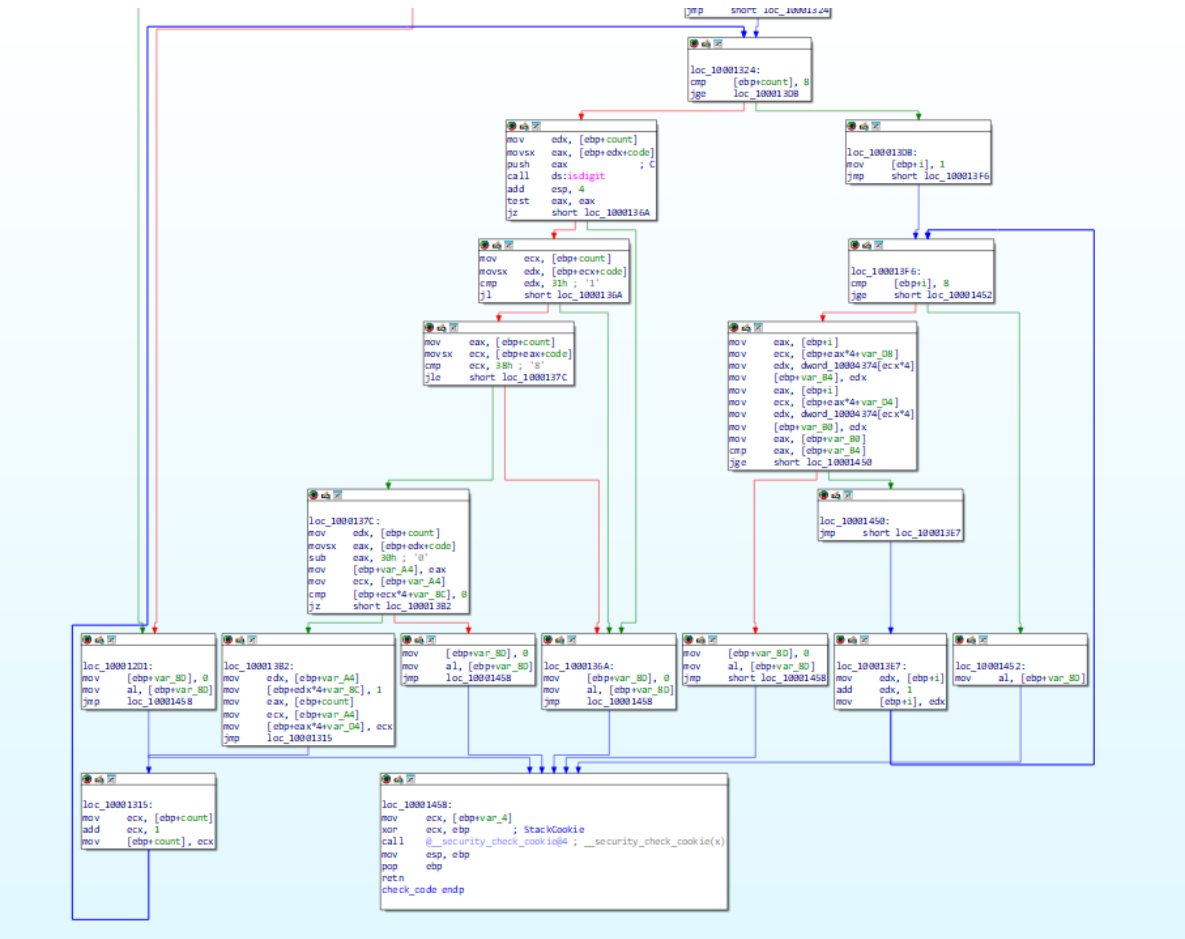
```
Stage 3: Welcome cyber specialist.
Your mission : Penetrate the security system of the supreme leader.
The location of the enriched Uranium is stored there.
Your country depends on your skills. We COUNT on you. Good luck.
Enter code
```

In this task, I realized that I was supposed to test what code would result in an entry. I entered to investigate the function that checks the correctness of the code using static analysis. The function returns 0 if the test failed and 1 if the test passed.

test eax,eax - eax must be equal to 1 to jump to success in printing



First of all, looking at the function on the structure, I see that the function performs a series of tests and loops, and if one of them fails, it returns a result of 0 and stops execution.



I Press F5 to convert the code, which was more convenient for me to see the entire function.

```
IDA View-A   Pseudocode-B   Pseudocode-A   Hex View-1   Local Types
17  v10 = 1;
18  if ( sub_10001100("%s", &code) == 1 && (v5 = v13, v8 = &v13[strlen(&code)], v4 = v8 - v13, v8 - v13 == 8) )
19  {
20    memset(v11, 0, sizeof(v11));
21    for ( i = 0; i < 8; ++i )
22    {
23      if ( !isdigit(v13[i - 1]) || v13[i - 1] < 49 || v13[i - 1] > 56 )
24      {
25        v10 = 0;
26        return 0;
27      }
28      v6 = v13[i - 1] - 48;
29      if ( v11[v6] )
30      {
31        v10 = 0;
32        return 0;
33      }
34      v11[v6] = 1;
35      v1[i + 1] = v6;
36    }
37    for ( j = 1; j < 8; ++j )
38    {
39      v2 = dword_10004374[v1[j]];
40      v3 = dword_10004374[v1[j + 1]];
41      if ( v3 < v2 )
42      {
43        v10 = 0;
44        return 0;
45      }
46    }
47    return v10;
48  }
49  else
50  {
51    v10 = 0;
52    return 0;
    }
00000663 check_code:17 (10001263)
```

The function performs the following checks:

1. String length check:

The string must be exactly 8 characters long.

2. Digit check:

Each character in the string must be a digit between 1 and 8 (inclusive).

3. Repetition check:

Each character in the string can only appear once (no duplicates).

4. Ascending order check:

The values in the string must be in ascending order based on the values in the dword_10004374 array. That means each character after the previous one must be equal to or greater than the one before it, according to the values in the array.

If all checks pass, the function returns 1. If any check fails, the function returns 0 and terminates execution immediately.

So first thing is values from 1-8 now the question is in what order

I look at the file that I entered values into in the previous step and convert its values to decimal to know what the ascending order is so that the test will pass successfully.

1E9B = 7,835

DE2A = 56,874

9371 =37,745

2035 =8,245

C952 = 51,538

C637 = 50,743

0C2E = 3,118

C1DC = 49,628

0C2E 1E9B 2035 9371 C1DC C637 C952 DE2A

This way the values are arranged in ascending order, so I thought the code that would lead to success would be: 71438652

But it gave an error The code is incorrect

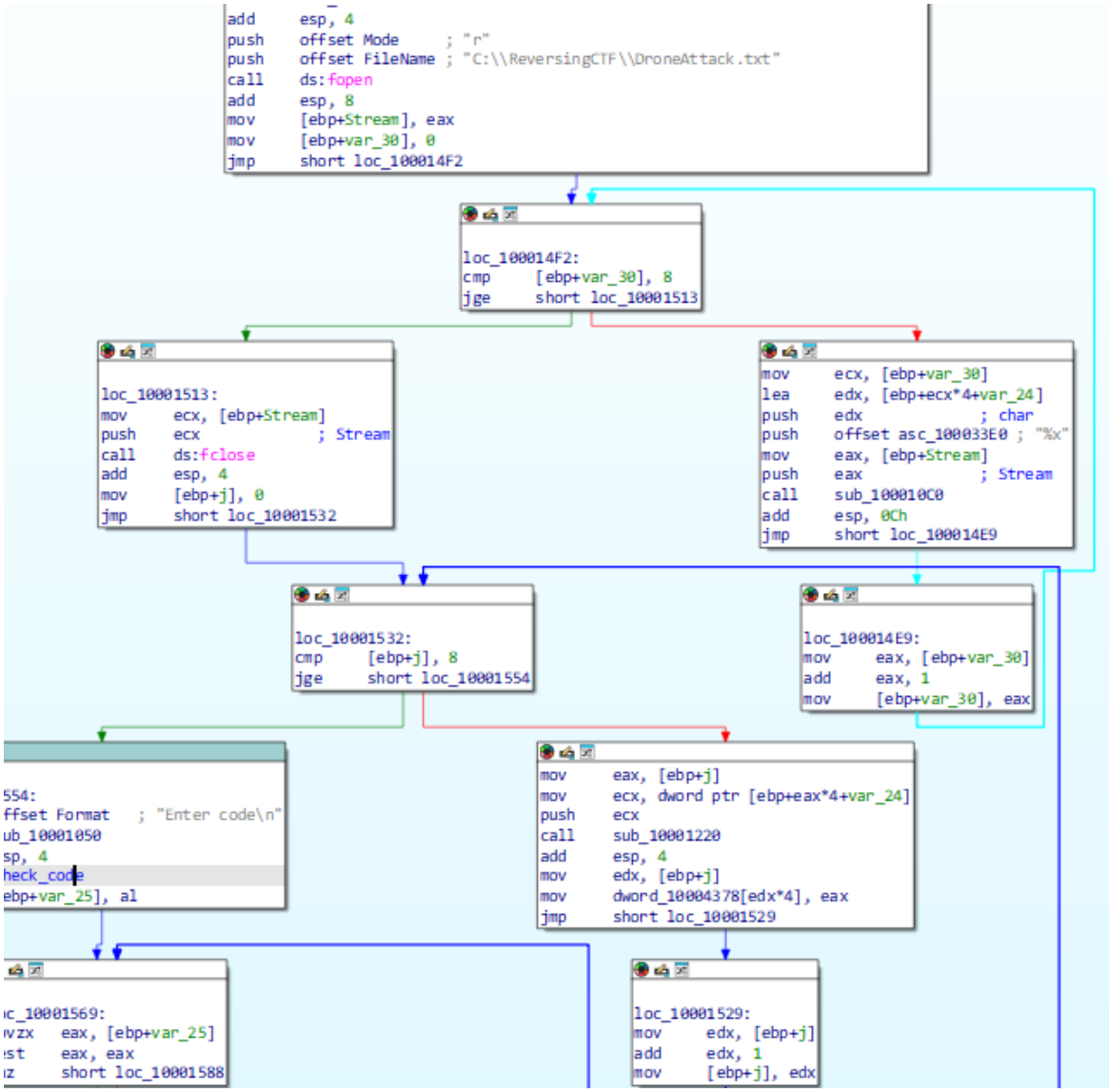
I tried all sorts of codes but it didn't work.

I thought I would arrange the file so that the values were arranged in ascending order and then I would enter the code 12345678 - But it didn't work.

I thought about trying to brute force the code, but there are many possibilities! That's 8! That's 40,320, and it would take time, so I'd better use my brain.

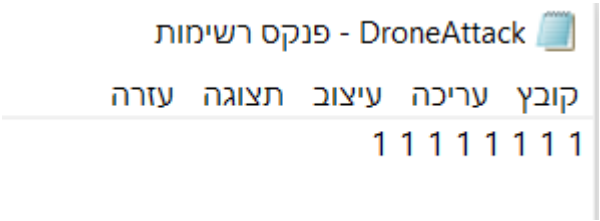
So I thought there was a change in the values in the file and yes I was right there is code that makes changes to the file.

Two loops that iterate over the file value 8 times, the number of values in it.

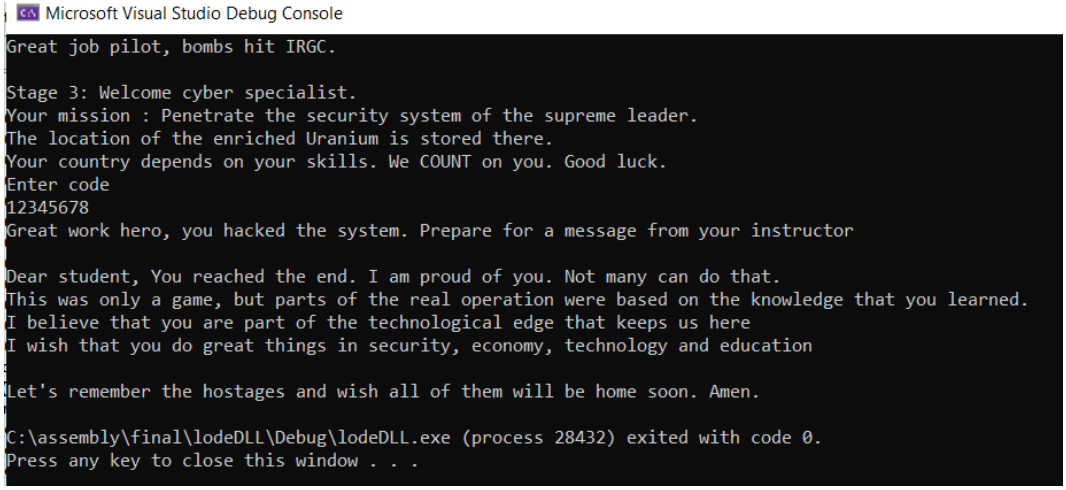
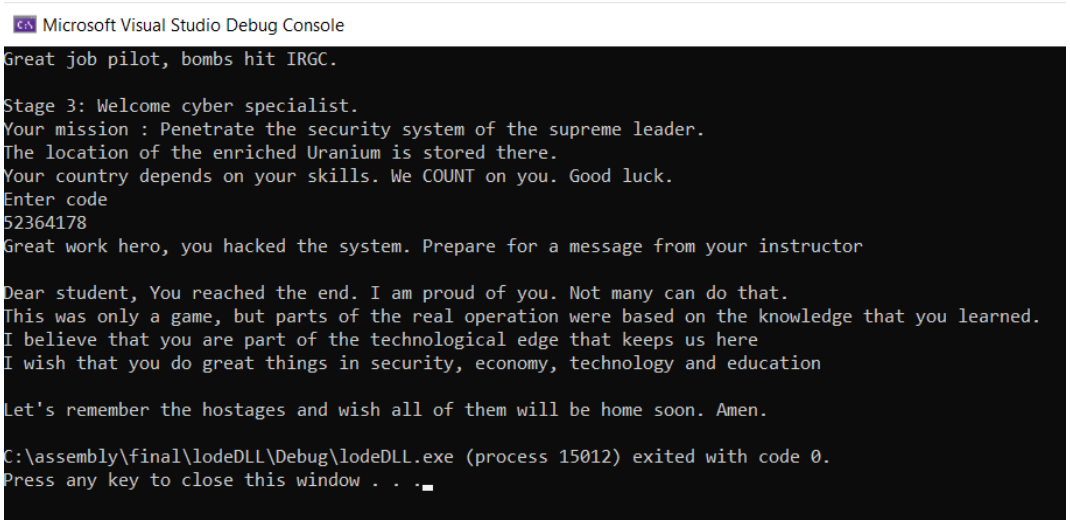


I had a hard time understanding what change was made to the file in static analysis.

Then I thought maybe I would completely change the values in the file that would contain the same value 8 times so that the order wouldn't change so it was worth a try. I put the value 1 (of course you can put any value) in the file 8 times.



I entered the code 12345678 which meets all the conditions of the tests: an 8-digit code with values 1-8 without duplicates and now after the change in the file the last condition that checks that it was OK is meaningless because all the values are equal.



And the code is correct, so you can put any code sequence that meets the following conditions, with 8 digits and values between 1-8 without duplicates.